

P0 구현 보고서 (실주문 없이 실계정 주문 파라미터 검증 파이프라인)

- 작성일: 2026-02-24
- 대상: 기획/개발 공용
- 프로젝트: `F:\src\Wquant-trading`
- 기준 문서:
 - `후속\_작업\_리스트업.pdf`
 - `P0\_실주문\_없이\_실계정주문\_파라미터\_검증\_파이프라인\_완성.pdf`

1) P0 목표

실주문 이전 단계에서 주문 파라미터/제약/매등/복구를 검증해, `REAL` 전환 시 중복주문/상태 꼬임 사고 확률을 낮추는 것이 목표다.

2) 구현 요약

- 주문 검증 API 추가: `orders/test`, `orders/chance`
- 주문 사전 Validation 레이어 추가: 최소 주문금액, 호가단위 보정, 포맷 정규화
- 매등키(`client\_order\_id`)와 업비트 식별자(`upbit\_identifier`) 분리
- 재전송 중심 로직 제거, 조회 복구(identifier 조회) 우선
- 거래 모드 확장: `REAL`, `TEST`, `PAPER`, `SHADOW`
- 테스트/실패 감사 로그 저장 필드 추가
- 오류 분류 체계 및 알림 메시지 표준화
- 관련 단위 테스트 보강

3) P0 티켓별 반영 상태

P0-01. `orders/test` 엔드포인트 연동

- 상태: 완료
- 구현:
 - `UpbitClient.test\_order(...)` 추가
 - `/v1/orders/test` 호출 구현(auth 포함)
- 파일:
 - `trader/exchange/upbit\_client.py`
 - `trader/trading/execution.py` (`trade\_mode=TEST` 경로)

P0-02. 주문 가능 정보(chance) 조회 + 캐시

- 상태: 완료
- 구현:
 - `UpbitClient.get\_order\_chance(...)` 추가
 - 인메모리 TTL 캐시(`\_chance\_cache`, 기본 900초)
- 파일:
 - `trader/exchange/upbit\_client.py`

P0-03. 주문 사전 Validation 레이어

- 상태: 완료
- 구현:
 - `ExecutionEngine.validate\_order\_params(...)` 추가
 - 최소 주문금액(`min\_total`) 미만 차단

- 호가단위 보정(기본 KRW 구간 규칙 + chance 우선)
- Decimal -> string 정규화
- 검증 실패 시 `REJECTED` + `VALIDATION\_ERROR`
- 파일:
 - `trader/trading/execution.py`
 - `trader/trading/error\_handling.py`
 - `tests/test\_execution\_and\_paper.py`

P0-04. `identifier` 재사용 불가 정책 반영

- 상태: 완료
- 구현:
 - `orders` 테이블에 `upbit\_identifier` 컬럼 추가
 - 내부 먹등키(`client\_order\_id`)와 업비트 식별자 분리
 - 업비트 식별자는 1회성 UUID 기반 생성
- 파일:
 - `trader/data/models.py`
 - `trader/data/db.py` (lightweight migration)
 - `trader/trading/execution.py`

P0-05. Submit 재시도 로직 교정(재전송 금지, 조회 복구 우선)

- 상태: 완료
- 구현:
 - 실주문은 1회 submit 시도
 - 실패 시 `get\_order\_by\_identifier(...)`로 복구 시도(백오프)
 - 복구 실패 시 `ERROR\_NEEDS\_REVIEW` 종료
 - 자동 재전송 금지
- 파일:
 - `trader/trading/execution.py`

P0-06. commit 누락 버그 정리

- 상태: 완료
- 구현:
 - 상태 변경 후 `self.session.commit()` 호출 흐름 정리
 - 오류/재시도/복구 경로에서 DB 반영 일관성 확보
- 파일:
 - `trader/trading/execution.py`

P0-07. 주문 테스트 모드 플래그 추가

- 상태: 완료
- 구현:
 - `trade\_mode` 도입: `REAL/TEST/PAPER/SHADOW`
 - `TEST`: `/v1/orders/test`만 호출
 - `SHADOW`: 검증/기록만, 전송 없음
 - 기존 `PAPER` 유지
- 파일:
 - `trader/config/settings.py`
 - `trader/trading/scheduler.py`

- `trader/trading/execution.py`
- `README.md`

P0-08. 테스트 주문 결과 저장(감사 로그)

- 상태: 완료
- 구현:
 - `orders.exchange\_response\_raw`, `orders.error\_class`, `orders.last\_error` 저장
 - TEST/REAL 모두 응답/오류를 원장에 기록
- 파일:
 - `trader/data/models.py`
 - `trader/data/db.py`
 - `trader/trading/execution.py`

P0-09. 리컨실 강화(open orders/accounts 기반)

- 상태: 완료(보강)
- 구현:
 - 기존 계좌/미체결 리컨실 유지
 - 미체결 매칭 키에 `upbit\_identifier` 추가 반영
 - 재기동/동기화 시 주문 정합성 보강
- 파일:
 - `trader/trading/reconcile.py`
 - `trader/trading/scheduler.py`

P0-10. 에러 분류 체계/알림 템플릿

- 상태: 완료
- 구현:
 - 오류 분류: `VALIDATION\_ERROR`, `RATE\_LIMIT`, `AUTH\_ERROR`, `NETWORK\_TIMEOUT`, `UNKNOWN`
 - 스케줄러 알림에 `market/side/price/volume/mode/error\_class/action/message` 포함
- 파일:
 - `trader/trading/error\_handling.py`
 - `trader/trading/scheduler.py`

4) 데이터 모델/마이그레이션 변경

- `orders` 신규/보강 컬럼:
 - `upbit\_identifier`
 - `error\_class`
 - `exchange\_response\_raw`
 - (`retry\_count`, `last\_error` 포함)
- lightweight migration 반영:
 - SQLite 기존 DB에 `ALTER TABLE`로 컬럼 추가
- 파일:
 - `trader/data/models.py`
 - `trader/data/db.py`

5) 테스트 결과

- 실행 일시: 2026-02-24

- 명령: `python -m pytest -q`
- 결과: `10 passed`

P0 관련 주요 테스트:

- 동일 먹등키 중복주문 방지
- 최소 주문금액 미만 사전 차단 + submit 미호출
- TEST 모드에서 `/v1/orders/test`만 호출 보장
- PAPER 체결 반영 1회성 검증

6) 기획/운영 관점 체크포인트

- `REAL` 전환 전 권장:
 1. `TRADE\_MODE=TEST`로 대상 마켓/전략 파라미터 검증 로그 확보
 2. `TRADE\_MODE=SHADOW`로 주문 의도 빈도/리스크 알림 검증
 3. 이후 초소액 `REAL` 전환
- 운영 중 확인 지표:
 - `orders.state` 분포(`TEST\_OK`, `REJECTED`, `ERROR\_NEEDS\_REVIEW`)
 - `orders.error\_class` 분포
 - 동일 봉 중복 주문 여부(`client\_order\_id` 기준)

7) 남은 후속 작업(P1 이상)

- 실계정 소액 E2E 리허설(주문 -> 체결 -> 재시작 복구)
- Shadow mode 장기 관찰(3~7일) 후 REAL 전환
- PnL 하드스탑 정밀화, 운영 대시보드 고도화